实验原理：哈希长度扩展攻击和碰撞攻击

0. 概述

哈希函数把任意长度的输入压成固定长度的摘要：

```
H: {0,1}* → {0,1}^n
```

密码学哈希要满足三条性质：

性质	含义	被破坏的后果
单向性	给定摘要，难以找回原消息	口令、承诺值泄露
抗第二原像	给定 m，难以找到 m′ ≠ m 使 H(m′) = H(m)	文件替换
抗碰撞	难以找到任意一对 m ≠ m′ 使 H(m) = H(m′)	证书、软件分发被替换

本实验的两种攻击：

1. **长度扩展**：不需要碰撞，也不需要密钥，只要知道 `H(secret || m)` 和 `len(secret)`，就能算出 `H(secret || m || padding || suffix)`。
2. **碰撞**：找到两个不同消息，摘要相同。MD5 的碰撞已经可以实用化。

两种攻击的根源不同：

- 长度扩展来自 **Merkle–Damgård 结构**；
- MD5 碰撞来自 **MD5 压缩函数本身的设计缺陷。**

---

1. 哈希函数

1.1 三个性质

单向性：给定 `h = H(m)`，很难求出 `m`。这里的「很难」指计算上不可行，不是信息论上不可能。

抗第二原像：给定 `m`，很难找到另一个 `m'`，使 `H(m') = H(m)`。

抗碰撞：很难找到任意一对不同的 `m, m'`，使 `H(m) = H(m')`。

抗碰撞比抗第二原像更强。能找碰撞，不一定能给定消息找第二原像；但抗碰撞被破坏，哈希就不能再用于需要「唯一性」的场合。

1.2 输出长度与生日攻击

如果哈希输出 n 位，朴素暴力找碰撞需要约 `2^(n/2)` 次计算，这叫生日攻击。

算法	输出	生日攻击复杂度	现状
MD5	128 位	约 2^64	碰撞已实用化
SHA-1	160 位	约 2^80	碰撞已实用化（2017）
SHA-256	256 位	约 2^128	仍安全
SHA-3	可变	约 2^(n/2)	仍安全

2^64 听起来很大，但 MD5 的实际碰撞攻击远低于这个数。2004 年王小云等人的攻击把复杂度降到了可计算的范围。

1.3 哈希不是加密

哈希没有密钥，不能「解密」。它只做单向压缩。

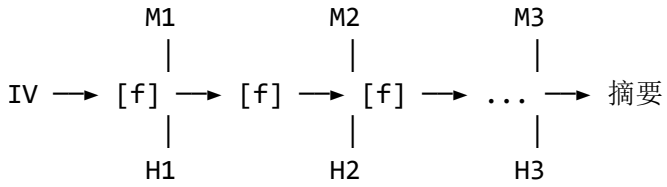
需要「带密钥的完整性」时，要用 MAC（消息认证码），例如 HMAC。

---

2. Merkle–Damgård 结构

2.1 分块与链式压缩

MD5、SHA-1、SHA-256 都采用 Merkle–Damgård 结构：



- 消息先补到块长的整数倍；
- 每块和当前状态一起送入压缩函数 `f`；
- 最后一块输出的状态就是摘要。

MD5 的块长是 512 位（64 字节），状态是 128 位（16 字节）。SHA-256 的块长也是 512 位，状态是 256 位。

这个结构有一个直接后果：**摘要就是处理完最后一块后的内部状态。**只要知道这个状态，就能从中间继续算。

2.2 填充规则

MD5 的填充：

```
0x80 | 若干 0x00 | 消息比特数（64 位，小端序）
```

填充后总长度必须是 64 的倍数。零的个数由下式决定：

```
zero_count = (56 - (msg_len + 1) % 64) % 64
```

为什么是 56：最后 8 字节要放长度，`56 + 8 = 64`。

几个边界：

消息长度	填充长度	说明
0	64	一整块
55	9	`0x80` + 8 字节长度，不加零
56	72	多出一整块
64	64	正好一块，仍要补一整块
119	9	与 55 同理
120	72	与 56 同理

填充**总是至少加 1 字节**，即使消息长度已经是 64 的倍数。这一点经常被理解错误，认为可以不填充。

2.3 长度字段

最后 8 字节是**消息长度的比特数**，不是字节数，也不是填充后的长度。

- 20 字节消息 → 160 比特 → `a0 00 00 00 00 00 00 00`（小端序）。
- 长度按 64 位取模，超过 `2^64` 比特的部分回绕。

小端序意味着最低有效字节在前。手算时先写成 16 进制，再按字节倒过来。

---

3. 长度扩展攻击

3.1 攻击目标

服务端用下面这条规则做消息认证：

```
tag = H(secret || message)
```

攻击者知道：

- tag；
- message；
- len(secret)（或者能猜到一个范围）。

攻击者不知道：

- secret 的内容。

目标是构造 message’ 和 tag’，使：

```
tag’ = H(secret || message’)
```

并且 message’ 包含攻击者想追加的内容。

3.2 为什么攻击可行

MD5 处理 secret || message 时，会先补填充，再逐块压缩：

```
secret || message || padding(secret || message)
```

处理完这些块之后，内部状态正好是 tag。

攻击者把这个状态作为初始状态，继续压缩 suffix：

```
tag’ = MD5_from_state(tag, suffix)
```

因为压缩函数只依赖「当前状态 + 下一块」，所以：

```
MD5_from_state(tag, suffix)
= H(secret || message || padding || suffix)
```

攻击者不需要知道 secret，只需要知道**填充有多少字节**。填充长度由 len(secret) + len(message) 决定，所以只要知道 len(secret) 就够了。

3.3 攻击者交出去的消息

攻击者不能只发 suffix。服务端会重新计算 H(secret || message’)，所以 message’ 必须包含：

```
message || padding || suffix
```

其中 padding 是 MD5 对 secret || message 的填充。这些字节里有 0x80 和 0x00，在 URL 里要做百分号编码。

服务端收到后：

1. 用自己的密码和收到的消息重新算哈希；
2. 因为消息里已经带了正确的填充和 suffix，算出来的结果等于 tag′；
3. 校验通过。

3.4 一个具体的长度计算

假设：

```
len(secret) = 8
len(message) = 47
```

那么 secret || message 是 55 字节。填充长度：

```
(56 - (55 + 1) % 64) % 64 = 0 个零
```

加上 0x80 和 8 字节长度，填充一共 9 字节。

新消息：

```
47 字节原消息 + 9 字节填充 + 24 字节 suffix = 80 字节
```

count（已经处理的比特数）是：

```
(8 + 47 + 9) * 8 = 512 比特
```

正好 8 个块？不，512 比特 = 64 字节 = 1 个块。原消息加密码 55 字节，填充 9 字节，正好 64 字节。

3.5 不知道密码长度怎么办

长度扩展需要填充长度。如果不知道 len(secret)，可以枚举常见长度：

```
for L in range(0, 65):
    candidate = forge(tag, message, L, suffix)
    试一下 candidate 能不能通过
```

服务端会告诉你哪一条通过。每试一次成本很低。

所以「密码长度保密」不能阻止长度扩展。它只是让攻击多几步。

3.6 哪些构造会中招

会中招：

```
H(secret || message)
H(secret || message || padding)        # 仍然可能
```

不会中招：

```
HMAC(secret, message)
H(message || secret)                   # 不容易做长度扩展，但另有问题
H(secret || message || secret)         # 不是标准构造，安全性没有保证
```

`H(message || secret)` 虽然不容易长度扩展，但容易受到「碰撞 + 后缀」一类攻击，也不是 MAC 的标准做法。不要自己发明。

3.7 长度扩展与碰撞的区别

项	长度扩展	碰撞
需要密钥吗	不需要知道密钥，只需长度	不需要密钥
攻击目标	在已知消息后追加内容	让两条不同消息摘要相同
受影响构造	`H(secret‖m)`	任何用该哈希做唯一标识的地方
防御	HMAC、AEAD	换抗碰撞哈希（SHA-256/3）

4. MD5 碰撞

4.1 什么是碰撞

碰撞是：

```
A ≠ B，但 MD5(A) = MD5(B)
```

理想情况下，找一对碰撞需要约 2^64 次计算。MD5 的实际碰撞攻击远低于这个数。

4.2 王小云 2004 年的结果

2004 年，王小云、冯登国、来学嘉、于红波公布了 MD5、MD4、HAVAL-128、RIPEMD 的碰撞。MD5 的碰撞抗性从此不被信任。

本实验使用的一对 128 字节碰撞块：

```
A: d131dd02c5e6eec4...
B: d131dd02c5e6eec4...
```

两个文件都是 128 字节，只有 6 个字节不同，差异一共 6 个比特：

偏移	A	B	异或
19	0x87	0x07	0x80
45	0x87	0x07	0x80
59	0x87	0x07	0x80
83	0x87	0x07	0x80
109	0x87	0x07	0x80
123	0x87	0x07	0x80

两块的 MD5 都是：

```
79054025255fb1a26e4bc422aef54eb4
```

SHA-256 则完全不同。

4.3 碰撞为什么能保留后缀

MD5 处理 A 和处理 B，经过相同的两块之后，得到**相同的内部状态。**

如果后面接相同的后缀：

```
A || suffix
B || suffix
```

那么从相同的状态出发，压缩同样的后缀块，结果当然相同。

如果前面加相同前缀：

```
prefix || A
prefix || B
```

前缀处理完后，进入 A/B 的起始状态相同；但 A 和 B 的内部字节不同，压缩后状态通常不同，后续就无法保证碰撞。实验中的具体前缀会让碰撞消失。

这就是「碰撞是起始状态 + 消息的性质」：

```
相同前缀 + 碰撞对          → 通常不再是碰撞
碰撞对 + 相同后缀          → 仍然是碰撞
```

4.4 碰撞攻击带来哪些问题

软件分发：下载站只校验 MD5。攻击者上传 evil，页面显示 good 的 MD5。用户下载 evil，校验通过。

证书：CA 只对摘要签名，不检查文件内容。攻击者让合法证书和恶意证书摘要相同，CA 签了合法证书，恶意证书也能用同一个签名。

版本控制：用 MD5 做对象 ID，两个不同对象会冲突。

4.5 真实事件

- **2008 年 Rogue CA**：研究者用 MD5 碰撞构造了一个伪造的 CA 证书。证书链的签名仍然有效，因为被签的是摘要。
- **2012 年 Flame**：恶意软件使用 MD5 碰撞伪造了 Microsoft 的代码签名证书。
- **2017 年 SHAttered**：Google 和 CWI 公布了 SHA-1 的第一对实用碰撞，两个 PDF 的 SHA-1 相同，内容不同。

这些事件说明：**哈希算法的碰撞抗性一旦被攻破，依赖它的整条信任链都会受影响。**

4.6 相同前缀碰撞与选择前缀碰撞

- **相同前缀碰撞**（identical-prefix collision）：A 和 B 的内容大部分相同，只有少数字节不同。本实验的碰撞块属于这一类。
- **选择前缀碰撞**（chosen-prefix collision）：攻击者可以指定两个不同的前缀，生成两段不同的碰撞块，使 `prefix1 || block1` 和 `prefix2 || block2` 碰撞。Flame 用的是这一类。

选择前缀碰撞更难，但一旦能做到，攻击场景更灵活。

---

5. 校验和的局限

5.1 校验和证明什么

非密钥校验和只能证明：

```
收到的字节，算出来的摘要，和发布者给的摘要一样
```

它不能证明：

```
这些字节是发布者当时发布的那一份
```

如果攻击者能让两个文件摘要相同，校验和会放行两个文件。

5.2 什么时候可以用 MD5

- 检测传输错误、磁盘坏块：可以，但更推荐 SHA-256。
- 对抗主动攻击者：不可以。
- 软件下载页：不可以只给 MD5。至少要给 SHA-256，最好再加签名。
- 口令存储：不可以直接用 MD5。要用 bcrypt、scrypt、Argon2 等带盐、慢速的算法。

5.3 换 SHA-256 就够了吗

不够。要看你用它做什么：

用途	只换 SHA-256 够不够
抗碰撞的文件校验	够（目前）
`H(secret‖m)` 做 MAC	**不够**，长度扩展仍然成立
口令存储	**不够**，需要慢哈希 + 盐
数字签名	够（配合标准签名方案）

长度扩展不依赖 MD5 的具体缺陷。SHA-256 也是 Merkle–Damgård，`SHA-256(secret || m)` 同样可以被长度扩展。换成 SHA-256 只解决了碰撞问题，没有解决拼接顺序问题。

---

6. HMAC

6.1 构造

HMAC 的定义：

```
HMAC(K, m) = H((K ⊕ opad) || H((K ⊕ ipad) || m))
```

其中：

- `ipad` 是 `0x36` 重复块长次；
- `opad` 是 `0x5c` 重复块长次；
- `K` 先补到块长（或先哈希，如果太长）。

6.2 为什么能抗长度扩展攻击

攻击者拿到 `HMAC(K, m)`，想续写 `m || padding || suffix`。但 HMAC 的输出是**外层哈希**的结果，不是内层哈希的状态。

外层哈希的输入是：

```
(K ⊕ opad) || 内层摘要
```

攻击者不知道 K，也算不出 K ⊕ opad，所以无法构造外层哈希的中间状态。长度扩展需要「知道某个中间状态」，而 HMAC 不把内层状态暴露出来。

6.3 为什么不用 H(K || m || K)

这种构造看起来把密钥放在两头，但没有标准安全性证明，容易受到长度扩展的变种攻击。不要自己设计 MAC。

6.4 比较 tag

校验 tag 时不要写：

```
if tag == expected:
    ...
```

字符串比较会在第一个不同字节处返回，攻击者可以通过计时猜出正确前缀。用：

```
hmac.compare_digest(tag, expected)
```

它按固定时间比较，不因为提前发现不同而短路。

6.5 HMAC 的边界

HMAC 不是万能的：

- 它不提供加密，只提供完整性和认证；
- 它需要双方共享密钥；
- 密钥管理仍然是大问题；
- 它不能防止重放攻击，需要 nonce、时间戳或序列号。

---

7. 本实验的代码结构

7.1 md5_soft.py

纯 Python 实现的 MD5，接口与 hashlib 类似，但多两个参数：

```
md5(data=b"", state=None, count=0)
```

- `state`：16 字节的中间状态；
- `count`：已经处理过的比特数。

正常哈希时两个参数都不传；长度扩展时传旧摘要和旧长度。

7.2 任务一

实现 `padding()` 和 `extend_digest()`，验证：

```
extend(H(m), len(m), suffix) == H(m || padding || suffix)
```

7.3 任务二

把任务一的能力用到银行 API：

```
token = MD5(password || message)
```

伪造：

```
message’ = message || padding || &command3=UnlockAllSafes
token’   = MD5_from_state(token, message’, count)
```

7.4 任务三

把 Wang 2004 碰撞块放在 zipapp 前面：

```
good = collision_a || payload
evil = collision_b || payload
```

`payload` 读取自己文件的第 `0x13` 字节，决定打印哪句话。

7.5 任务四

对比：

```
naive = SHA-256(secret || message)
mac   = HMAC-SHA-256(secret, message)
```

并用 `hmac.compare_digest` 校验。

---

9. 思考题

1. MD5 的块长、状态长度、摘要长度各是多少？
2. 一条 56 字节的消息，MD5 填充多少字节？为什么？
3. 长度扩展需要知道密钥的哪些信息？
4. 为什么 `H(secret || m)` 可以被长度扩展，而 HMAC 不可以？
5. 两个文件 MD5 相同，SHA-256 不同，说明什么？
6. 在碰撞块前面加相同前缀，为什么碰撞通常会消失？
7. 只把 MD5 换成 SHA-256，能不能修复 `H(secret || m)` 的长度扩展问题？
8. 为什么比较 MAC 要用 `compare_digest`？

> 思考题不要求提交，只给大家在实验过程中思考，让大家更好地理解实验内容。